

Extract Discriminative Features: Profiled Side-Channel Analysis for Cryptosystems Based on Supervised Contrastive Learning

Zoushaojie Jiang, An Wang, Yaoling Ding, Annv Liu, Zheng Liu, Jing Yu, and Liehuang Zhu *Senior Member, IEEE*

Abstract—Deep learning-based profiled side-channel analysis (SCA) targeting cryptographic implementations has attracted significant attention in recent years. Generalizable deep learning mechanisms, such as contrastive learning-based profiled SCA (CL-SCA), can enhance the effectiveness of SCA without reliance on specific network architectures and hyperparameters. This independence enables robust adaptation across diverse attack scenarios. Nonetheless, CL-SCA relies heavily on data augmentation and may mistakenly push apart physical leakage traces that should belong to the same class, which interferes with the extraction of discriminative features crucial for SCA performance. To address these limitations, we propose a profiled SCA method based on supervised contrastive learning, called SupCL-SCA. This method enhances the learning of discriminative features that facilitate key recovery by leveraging supervised information to guide the extraction of similarities in feature space. Compared with state-of-the-art methods, SupCL-SCA not only retains their general applicability and inherent advantages but also eliminates reliance on complex data augmentation and multi-stage training. Additionally, we propose a cosine distance-based Intra-Inter Distance Ratio (IIDR) metric to assess the discriminative capability of models in deep learning-based profiled SCA methods. We evaluate SupCL-SCA on three publicly available datasets covering different implementations and platforms. Experimental results show that SupCL-SCA consistently reduces the number of traces required to recover the key compared to the original methods, demonstrating enhanced attack capability.

Index Terms—Side-channel analysis, Supervised contrastive learning, Neural networks, Profiled analysis, Cryptosystems.

I. INTRODUCTION

Side-channel analysis (SCA) has emerged as an effective physical security attack method, leveraging physical leaks such as electromagnetic signals and energy consumption generated during the operation of cryptographic devices to infer sensitive key variables of the cryptographic chip [1]–[5].

Profiled SCA is one of the most powerful forms of SCA [6], [7]. It assumes that the attacker has access to a profiled device that is nearly identical to the target device and has write access to the key. By collecting a large number of physical leakage traces from the profiled device, the attacker can accurately characterize the leakage features, construct a template, and

further recover the key. Nevertheless, traditional profiled SCA methods, such as template attacks, typically rely on multivariate Gaussian models to characterize the leakage distribution. These methods are not effective in handling high-dimensional traces and are unable to cope with countermeasures such as random delays [8].

In response to these challenges, researchers have begun exploring the integration of machine learning techniques into the field of SCA. Hospodar et al. [9] proposed treating profiled SCA as a classification problem and suggested replacing traditional statistical modeling methods with machine learning methods. Lerman et al. [10] demonstrated that machine learning methods could relax the parameterized assumptions about the distribution of trace data, allowing leakage features to be learned directly from large datasets.

In the context of SCA against masked implementations, deep neural networks, particularly Convolutional Neural Network (CNN), have become powerful tools for attacking due to their strong feature extraction capabilities [11]–[13]. Zaid et al. [14] proposed that using convolutional layers with a filter size of 1 enhances the network’s feature extraction ability, thereby improving attack performance under masking countermeasures. Cao et al. [15] proposed a bilinear CNN to incorporate the combination of leakage features explicitly into the network architecture for attacking masked implementations. Nevertheless, these methods focus on modifying network structures to bypass the limitations imposed by masking. Additionally, neural network architectures are highly dependent on the data distribution, and their performance may significantly degrade when the data changes.

Thus, recent researchers have shifted from focusing on optimizing model structures and hyperparameters to exploring more generalizable profiled SCA mechanisms based on deep learning, aimed at achieving effective adaptation and performance enhancement across various scenarios. Liu et al. [16] proposed a contrastive learning-based profiled SCA method, called CL-SCA. This method is not limited to specific network architectures, achieving robust classification results by capturing the similarities and differences between traces. However, under unsupervised conditions, the effectiveness of this method is influenced by the choice of data augmentation strategies.

To address this issue, we propose a supervised contrastive learning-based profiled SCA method, called SupCL-SCA. This method guides the extraction of similarity information with supervised information, steering the model to learn discriminative and more generalized feature representations while avoiding unnecessary steps like data augmentation. Furthermore,

This work is supported by National Key R&D Program of China (No. 2022YFB3103800), National Natural Science Foundation of China (Nos. 62272047, 62302036, 62402039), Beijing Natural Science Foundation (Nos. L244044, L251068).

Zoushaojie Jiang and An Wang contributed equally to this work (Equal contribution).

Zoushaojie Jiang, An Wang, Yaoling Ding, Annv Liu, Zheng Liu, Jing Yu and Liehuang Zhu are with the School of Cyberspace Science and Technology, Beijing Institute of Technology, Beijing 100081, China. (Email: {zoushaojiejiang, wanganl, dyl19, Annvliu, liuzheng98, yujing93, liehuangz}@bit.edu.cn).

(Corresponding authors: Yaoling Ding and Annv Liu).

SupCL-SCA does not replace existing network designs but serves as an enhancement strategy at the mechanism level. It can seamlessly integrate with various mainstream architectures (such as CNN), further improving the method's performance while inheriting the advantages of the original structure.

A. Related Works

Our work is related to a series of studies that use deep learning techniques to enhance the performance of profiled SCA. Maghrebi et al. [17] were the first to propose neural networks into profiled SCA. Cagli et al. [18] proposed combining CNN with data augmentation techniques to effectively perform SCA without preprocessing. In the context of masking countermeasures, Benadjila et al. [19] proposed the first publicly available dataset named ASCAD, providing a common foundation for deep learning-based profiled SCA applications.

Based on this public dataset, Jin et al. [20] began exploring how to build suitable neural network architectures to improve the performance of profiled SCA, incorporating the convolutional block attention module into the CNN architecture to allow the network to focus on the most relevant leakage points in the input traces. Zaid et al. [14] enhanced feature extraction with a convolutional layer of size 1 and simplified the network architecture with waveform preprocessing. In contrast, Wouters et al. [21] argued that a convolutional layer with a filter size of 1 primarily scales the input and could be replaced with preprocessing methods. Additionally, Cao et al. [15] embedded the product combination of masking leakage information into the neural network and proposed a bilinear convolutional neural network, called B-CNN, simplifying the learning task and significantly improving attack performance.

Although optimization of neural network structures had yielded some improvements in SCA, the performance gains were reaching a saturation point. Consequently, researchers began to explore new directions, such as the introduction of contrastive learning to further enhance performance. Liu et al. [16] proposed the CL-SCA method, which not only focuses on the label features of traces but also extracts similarities and differences between traces, improving key recovery performance while not being limited to specialized neural network structures. However, the CL-SCA method excessively relies on data augmentation strategies for extracting similarity information and may erroneously distinguish between samples of the same class, which negatively impacts the classification performance.

Therefore, the motivation for our work is to explicitly bring together samples of the same class in the feature space and extract discriminative features through supervised information. Meanwhile, SupCL-SCA integrates feature extraction and classification, jointly guiding the training of the model. SupCL-SCA does not replace existing neural network structures. Instead, it can seamlessly integrate into previous works, further enhancing the performance of existing methods.

B. Our Contributions and Paper Organization

The main contributions of this paper are summarized as follows:

- We propose a supervised contrastive learning-based SCA method, called SupCL-SCA. This method extracts discriminative features by explicitly aggregating samples of the same class, thereby enhancing the performance of SCA.
- The SupCL-SCA method does not rely on complex data augmentation or a two-phase training process. It inherits the advantage of CL-SCA in extracting trace similarities and differences to enhance key recovery performance, while significantly reducing training complexity.
- We propose an Intra-Inter Distance Ratio (IIDR) based on cosine distance as a similarity evaluation metric to measure the model's ability to aggregate similar samples and separate dissimilar ones in the feature space, offering a direct assessment of the model's discriminative capability.

The remainder of this paper is organized as follows: Section II provides background on CNN, deep learning-based profiled SCA and contrastive learning techniques. Section III analyzes the limitations of CL-SCA and presents the proposed method in detail, along with the IIDR evaluation metric. Section IV reports the experimental results. Finally, Section V concludes the paper.

II. PRELIMINARIES

A. Notation

In this paper, we use uppercase letters (e.g., X) to represent sets, and the corresponding lowercase letters to represent elements within those sets. The subscript of the lowercase letters denotes the index of the element in the set. For profiled SCA, we denote the set of traces as $X = (x_1, x_2, \dots, x_n)$, where x_i represents the physical leakage trace during the execution of the encryption operation, and y_i represents the label corresponding to x_i . We use $M = \{m_1, m_2, \dots, m_n\}$ to represent the set of plaintexts or ciphertexts corresponding to X , and $K = \{0, 1, \dots, 255\}$ to represent a set of one-byte key candidates.

We define the key used by the profiled device as k_p , the true key of the target device as k^* , and k as a key candidate. The function $\delta(\cdot)$ is the label generation function, which generates classification labels by combining the plaintext/ciphertext and the key. The classification is performed by a neural network model $f(\cdot)$, where the encoder part is denoted as $\varphi(\cdot)$. We define z as the feature vector in the feature space output by the encoder $\varphi(\cdot)$ from x , and $\Pr[K]$ as the probability distribution of K . The posterior probability of key candidate k given a trace x is represented as $\Pr[k|x]$.

B. Convolutional Neural Network

Profiled SCA is essentially a classification problem, and CNN have been widely applied to this task due to their advantages in feature extraction and pattern recognition. CNN is a feedforward neural network equipped with local connections and parameter sharing mechanisms, enabling efficient extraction of multi-variable features. Its architecture typically consists of convolutional layers, pooling layers, and fully connected layers, as shown in Fig. 1.

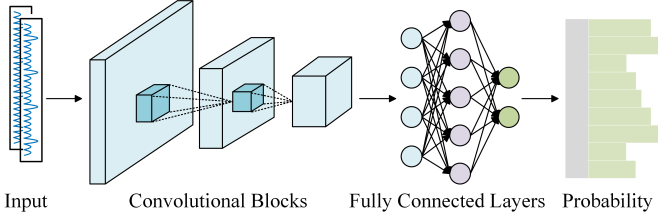


Fig. 1. The schema of CNN.

The convolutional layer is the core component of CNN, composed of multiple trainable convolutional kernels. Its basic operation involves sliding these kernels over the input data and performing convolution operations to extract local features at different scales and regions. The convolutional layer utilizes local connectivity and weight sharing mechanisms, effectively reducing the number of parameters in the network while enhancing the model's robustness to translation, scaling, and local deformations in the input.

The pooling layer typically follows the convolutional layer and serves as a downsampling layer to reduce the size of the feature maps. There are various types of pooling operations, typically involving selecting the maximum or average value within a local region to retain the most significant features, minimize redundant information, and improve the stability of the feature representation.

The fully connected layer is located at the end of the CNN, where each neuron is connected to all neurons in the previous layer. The high-dimensional feature maps output by the convolutional and pooling layers are typically flattened into a one-dimensional vector before entering the fully connected layer. The fully connected layer performs a global combination of the previously extracted local features and outputs the final prediction result.

C. Profiled SCA Based on Deep Learning

Profiled SCA based on deep learning has become one of the most promising SCA methods due to its independence from manual preprocessing and its ability for automatic feature extraction. Profiled SCA assumes that the attacker has access to a profiled device that is identical or similar to the target device. Building on this, deep learning techniques can directly learn an efficient mapping from leakage features to intermediate values from the raw traces collected by the profiled device, making it a key area of research in recent years.

Profiled SCA based on deep learning can be divided into two phases: the profiling phase and the analysis phase. In the profiling phase, the attacker selects plaintext m_i and key k_p , performs encryption operations on the profiled device, and collects N_p traces $X = (x_1, x_2, \dots, x_{N_p})$. Simultaneously, the corresponding plaintext or ciphertext $M = (m_1, m_2, \dots, m_{N_p})$ for each trace is recorded. The label for each trace $y_i = \delta(m_i, k_p)$ is computed and used to train the deep neural network f .

In the analysis phase, the attacker acquires N_a traces $X = \{x_1, x_2, \dots, x_{N_a}\}$ from the target device and utilizes the trained deep neural network f to compute the probability

vector for each trace x_i corresponding to the key candidates $k \in K$, as follows:

$$f(x_i) = \Pr[y = \delta(m_i, k) | x_i] = \Pr[k | x_i] \quad (1)$$

where $K = \{0, 1, \dots, 255\}$ denotes the complete key candidate space, and $\Pr[k | x_i]$ represents the posterior probability output by the network for each candidate key k given the input trace x_i .

A widely used evaluation metric in SCA is the guessing entropy (GE) [22], which is defined as the average rank of the correct key k^* over the key space K , as shown in equation (2):

$$\text{GE} = \text{rank} \left\{ \sum_{l=1}^Q \log_2 \Pr[k^* | x_i] \right\} \quad (2)$$

where $\text{rank}(\cdot)$ denotes the rank function. As the GE decreases, it indicates that the correct key k^* ranks higher on average within the entire key space K . When GE equals zero, it signifies the successful recovery of the correct key and indicates that the analysis was successful.

D. Supervised and unsupervised Contrastive Learning

Contrastive Learning is a representation learning method that trains models by constructing sample pairs. The core idea is to minimize the distance between similar samples while maximizing the distance between dissimilar samples, thereby enabling the model to learn more discriminative embedding representations in the feature space. In recent years, contrastive learning has demonstrated strong feature extraction capabilities and good transfer performance across multiple domains, such as image recognition and natural language processing. Based on the use of label information, contrastive learning can be roughly divided into two types: unsupervised contrastive learning and supervised contrastive learning.

Unsupervised contrastive learning does not rely on manual labels during training. Instead, it generates positive sample pairs by applying different data augmentations (e.g., cropping, transformation) to the input data, while other augmented samples are treated as negative samples [23]. This method typically employs an encoder to map the samples to high-dimensional vectors and utilizes contrastive loss functions such as information noise-contrastive estimation or normalized temperature-scaled cross entropy to optimize the embedding space structure without using labels, thereby uncovering discriminative features between samples.

Supervised contrastive learning utilizes true labels, treating samples of the same class as positive samples and explicitly pulling them closer, while samples from different classes are treated as negative samples, promoting their separation. This method results in better intra-class aggregation and inter-class separation in the embedding space [24]–[27]. As shown in Fig. 2, when a sample x_i is considered the anchor, samples of the same class (e.g., x_j and x_k) are treated as positive samples for anchor x_i , and samples from different classes (e.g., x_l and x_m) are treated as negative samples for x_i . The encoder $\varphi(\cdot)$ maps the samples to feature space vectors. By minimizing the similarity between positive samples and minimizing the similarity to negative samples in the feature

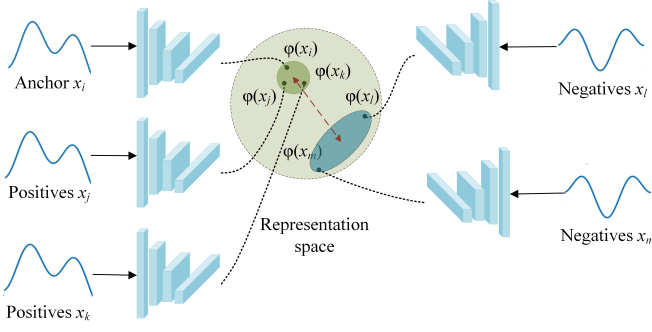


Fig. 2. The process of supervised contrastive learning

space, the model progressively builds a structure in high-dimensional space where the classes are clearly separable. This method not only increases the number of positive samples for each anchor but also improves the accuracy of the positive-negative sample classification, enabling the model to learn more compact intra-class aggregation and clearer inter-class separation in the feature space.

III. PROFILED SCA BASED ON SUPERVISED CONTRASTIVE LEARNING

A. The Limits of CL-SCA

Although CL-SCA [16] provides new insights for improving the feature extraction capability of profiled SCA, it still faces several limitations in terms of practicality:

1) **Negative Impact of Augmentation Strategy:** The CL-SCA method overly relies on data augmentation techniques to construct positive and negative sample pairs in order to capture the similarities and differences between various traces. However, the selection of augmentation strategies needs to be carefully designed according to the characteristics of the dataset. Inappropriate choices may affect the extraction of similarity information, degrade the model's performance, and significantly increase the complexity and resource consumption of hyperparameter tuning in experiments.

2) **Significant computational overhead:** The CL-SCA method employs a two-phase training process, which requires additional pretraining and random data augmentation to build positive and negative samples, leading to a substantial increase in computational overhead.

3) **Impact of Incorrect Sample Classification:** Due to the lack of label guidance in CL-SCA, when constructing negative samples, the model may mistakenly classify samples that actually belong to the same class as unrelated samples. This erroneous classification of samples interferes with the model's learning process, causing biases in distinguishing between similar and dissimilar samples, which in turn negatively affects the overall performance.

To address these issues, we leverage supervised information to guide the extraction of similarity information, thereby overcoming the limitations of CL-SCA and further improving the performance of deep learning-based profiled SCA.

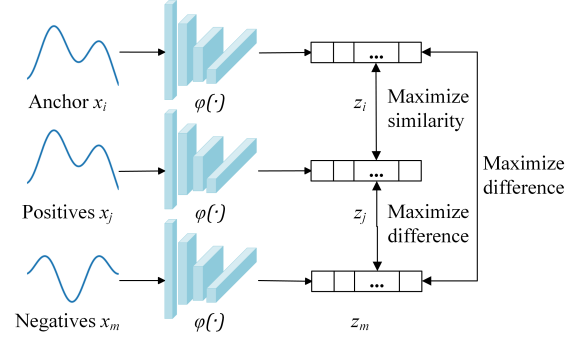


Fig. 3. The basic flow of Supervised Contrastive Loss

B. SCA Method Based on Supervised Contrastive Learning

Inspired by previous works [28], [29], we define $\varphi(\cdot)$ as the input feature vector of the classification layer in the CNN (if there is an activation function before the classification layer, the input before the activation is used). SupCL-SCA does not require the design of projection heads, encoders, or other network structures. Instead, it only requires replacing the loss function used in network training with the SupCL-SCA loss proposed in this paper, making it applicable to any network architecture.

Fig. 3 illustrates the construction mechanism of the supervised contrastive loss. Specifically, in SupCL-SCA, a trace x_i is treated as the anchor sample, and samples from the same class are regarded as positive samples, while samples from different classes are treated as negative samples. To guide the model in learning discriminative feature representations and computing the similarity between samples, we use $\|\cdot\|$ to represent ℓ_2 regularization and define the cosine similarity between two vectors α and β as follows:

$$\text{sim}(\alpha, \beta) = \frac{\alpha^T \beta}{\|\alpha\| \|\beta\|} \quad (3)$$

Given a mini-batch of N traces $\{x_1, x_2, \dots, x_N\}$, we treat traces from the same class as positive samples x_j and traces from different classes as negative samples x_m . Specifically, we define the sum of similarities between the positive samples of trace x_i as:

$$S_{x_i}^+ = \sum_{p \in P(i)} \exp\left(\frac{\text{sim}(z_i, z_j)}{\tau}\right) \quad (4)$$

where z_i is the feature vector of trace x_i obtained through $\varphi(\cdot)$, z_j is the feature vector of the positive sample x_j from the same class, $P(i)$ represents the set of indices of positive samples in the same mini-batch as x_i , and τ is a temperature scaling coefficient.

Similarly, we define the sum of similarities between the positive and negative samples of trace x_i as:

$$S_{x_i}^- = \sum_{k=1}^N \mathbb{I}_{[k \neq i]} \exp\left(\frac{\text{sim}(z_i, z_k)}{\tau}\right) \quad (5)$$

where z_k is the feature vector of trace x_k (for $k \neq i$) in the mini-batch, N is the size of the mini-batch, and $\mathbb{I}_{[k \neq i]}$ is an

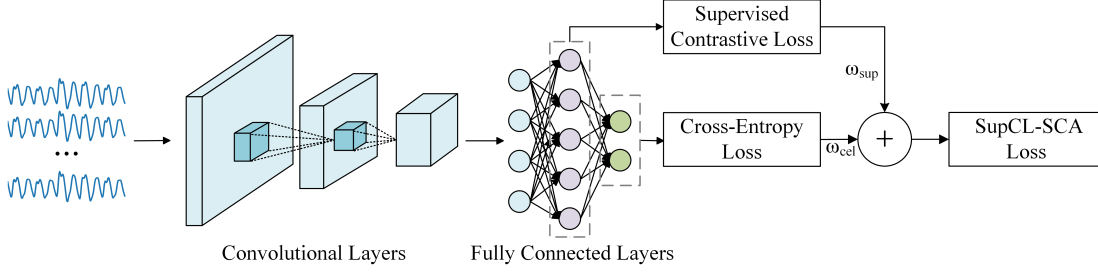


Fig. 4. The basic flow of SupCL-SCA

indicator function that equals 1 if $k \neq i$, and 0 otherwise. We define the loss function for trace x_i as:

$$\ell_{x_i} = -\log \left(\frac{S_{x_i}^+ + \epsilon \cdot \mathbb{I}_{[S_{x_i}^+=0]}}{S_{x_i}^-} \right) \quad (6)$$

where $\mathbb{I}_{[S_{x_i}^+=0]}$ is an indicator function that equals 1 if $S_{x_i}^+ = 0$, and 0 otherwise. ϵ is a constant. The term ϵ prevents the logarithm from being evaluated at zero, avoiding gradient explosion when no positive samples with the same label as trace x_i exist within the mini-batch.

The supervised contrastive loss ℓ_{sup} is defined as the average loss across all traces in the mini-batch, as shown in equation (7):

$$\ell_{\text{sup}} = \frac{1}{N} \sum_{i=1}^N \ell_{x_i} \quad (7)$$

In addition to the supervised contrastive loss ℓ_{sup} , we incorporate cross-entropy loss into the neural network parameter update process to improve the model's classification ability on key-related features. The cross-entropy loss is defined as:

$$\ell_{\text{cel}} = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{ic} \log p_{ic} \quad (8)$$

where p_{ic} represents the posterior probability distribution predicted for trace x_i , y_{ic} is the binary indicator function that equals 1 if the true class of sample i is class c , and 0 otherwise. N is the size of the mini-batch, and C is the number of classes.

Finally, as shown in Fig. 4, the SupCL-SCA loss we proposed is a weighted combination of the supervised contrastive loss ℓ_{sup} and the cross-entropy loss ℓ_{cel} :

$$\ell_{\text{sup-sca}} = w_{\text{cel}} \ell_{\text{cel}} + w_{\text{sup}} \ell_{\text{sup}} \quad (9)$$

where w_{cel} and w_{sup} are the weights for the supervised contrastive loss ℓ_{sup} and the cross-entropy loss ℓ_{cel} , respectively. These weights ensure that both losses are simultaneously optimized during training and balance their contributions to the overall network training process.

C. Evaluation of Model Discriminative Capability

In neural network-based classification tasks, the input feature vector of the network's classification layer is typically regarded as the discriminative feature of the sample, and its quality directly influences the classification performance. To assess the model's discriminative capability, we propose a

cosine distance-based similarity evaluation metric called the Intra-Inter Distance Ratio (IIDR). The cosine distance between two vectors α and β is defined as follows:

$$d(\alpha, \beta) = 1 - \text{sim}(\alpha, \beta) = 1 - \frac{\alpha^\top \beta}{\|\alpha\| \|\beta\|} \quad (10)$$

For each trace x_i , we compute its feature vector z_i through the encoder $\varphi(\cdot)$ and calculate the average cosine distance between z_i and its positive samples x_j , which reflects the aggregation of features within the same class. At the same time, we identify the negative sample class x_m with the smallest average cosine distance to x_i , and compute the average distance between z_i and the feature vector z_m of the negative sample class, which serves as the measure of inter-class separation.

Thus, we define $d_{\text{intra}}^{(i)}$ as the average cosine distance between trace x_i and its positive samples, and $d_{\text{inter}}^{(i)}$ as the average cosine distance between x_i and the negative sample class with the smallest average distance. The IIDR is then defined as:

$$\text{IIDR} = \frac{1}{N} \sum_{i=1}^N \frac{d_{\text{intra}}^{(i)}}{d_{\text{inter}}^{(i)} + \epsilon} \quad (11)$$

where ϵ is a small constant to avoid division by zero.

A lower IIDR indicates more compact intra-class features and clearer boundaries between key classes. This metric quantifies the degree of aggregation of samples within the same class and the separation between samples from different classes, revealing the model's discriminative ability in the latent feature space.

IV. EXPERIMENTS

To evaluate the effectiveness of the proposed method, experiments are conducted on several publicly available datasets in this section. All experiments are performed in a Python 3.9 environment, implemented using the PyTorch library, and executed on a server equipped with 128GB RAM and an NVIDIA GeForce RTX 4090 with 24GB memory.

To ensure fairness, we shuffle the attack trace order of each model in the analysis phase 100 times. The GE curve corresponding to the number of attack traces for each attack trace is then computed, and the average GE curve of the 100 trials is used as the final result. If the average GE curve eventually drops to zero, we report the minimum number of traces required to bring the average GE curve to zero, denoted as N_{GE} . For each experiment, 10 SupCL-SCA models are trained, and the average value is taken as the final result.

TABLE I
INFORMATION ABOUT DATASETS

Dataset	Implementation	SCA constraints	#Features
ASCAD ^{sync}	ATMega8515	First-order mask	700
ASCAD ^{desync100}	ATMega8515	First-order mask	700
AES_HD	Virtex-5 FPGA	None	1250
CHES CTF 2018	STM Microcontroller	First-order mask	2200

A. Datasets

In the experiments, we consider three public datasets, all corresponding to implementations of the AES algorithm. In Table I, we summarize the details of the datasets considered.

1) *ASCAD Fixed-Key Dataset*: The ASCAD dataset¹, proposed by Prouff et al. [19] in 2018, contains electromagnetic leakage collected from a masked AES implementation running on the 8-bit AVR architecture ATMega8515 microcontroller. The sampling rate is 2GS/s. The dataset consists of 60000 traces, with 50000 used for training and 10000 for testing. Each trace contains 700 sampling points and uses uniformly random plaintext and a constant key. The leakage model of ASCAD focuses on the third byte of the S-box output in the first round of the AES algorithm using the identity leakage model. The dataset has three variants with different levels of desynchronization, namely ASCAD^{sync}, ASCAD^{desync50}, and ASCAD^{desync100}. In this paper, we perform experiments on ASCAD^{sync} (desynchronization level 0) and ASCAD^{desync100} (desynchronization level 100).

2) *AES_HD Dataset*: The AES_HD dataset² was proposed by Picek et al. [30] in 2019 and contains electromagnetic leakage collected from an unprotected AES-128 implementation running on the Xilinx Virtex-5 FPGA. The dataset consists of 75000 traces, with each trace containing 1250 sampling points. Of these, 50000 traces are used for training, and 25000 are used for testing. The leakage model focuses on the leakage characteristics of the register overwrite in the last round of computation, using the identity leakage model. In this model, the leakage value is represented as the XOR result of the 11th byte of the ciphertext and the 11th byte of the key, followed by the inverse byte substitution operation, and then XORed with the 7th byte of the ciphertext.

3) *CHES CTF 2018 Dataset*: This dataset³, known as the CHES 2018 Capture-the-flag (CTF) AES-128 trace set, contains electromagnetic leakage collected from a masked AES-128 implementation running on a 32-bit STM microcontroller. The dataset includes 45000 training traces and 5000 attack traces, each containing 2200 sampling points. The training traces are collected from a device operating with a fixed key, whereas the attack traces are collected from the other device using a different key. In this paper, the leakage model focuses on the first byte of the S-box output in the first round of the AES operation, using the identity leakage model.

¹This dataset is publicly available at https://github.com/ANSSI-FR/ASCAD/tree/master/ATMEGA_AES_v1/ATM_AES_v1_fixed_key

²This dataset is publicly available at https://github.com/AESHD/AES_HD_Dataset

³This dataset is publicly available at <https://chesctf.riscure.com/2018/content?show=training>

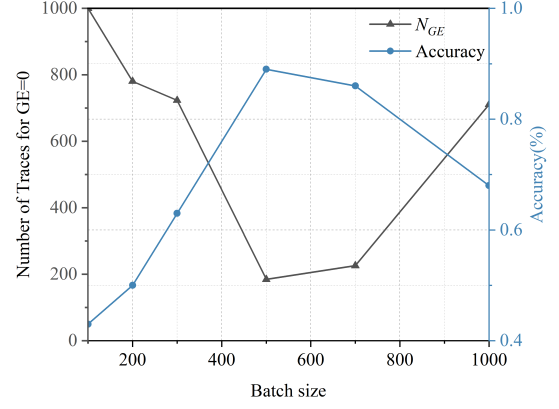


Fig. 5. Impact of batch size when w_{sup} is fixed

B. Experiments on batch size and weight

To investigate the impact of batch size and the weights w_{cel} and w_{sup} in $\ell_{sup-sca}$ on the attack performance of SupCL-SCA, we conduct experiments on the ASCAD^{sync} dataset, using the same network architecture and hyperparameters as in [21].

Batch size for SupCL-SCA. As discussed in Section III-B, for a mini-batch containing N traces $\{x_1, x_2, \dots, x_N\}$, each trace x_i is paired with other traces x_j of the same class as positive samples, and with traces x_m of a different class as negative samples. Therefore, batch size directly determines the number of positive and negative samples associated with each trace during training, which in turn has a significant impact on the model's final performance. The following experiments will investigate the impact of batch size on model performance.

Fig. 5 shows how N_{GE} and accuracy of the model changes with batch size when the weights are fixed. When the model fails to successfully attack within 1000 traces, we set the N_{GE} to 1000. Specifically, as the batch size increases, the N_{GE} exhibits a nonlinear trend, first decreasing and then increasing. When the batch size is 100, the N_{GE} value is 1000, indicating that the model cannot recover the correct key within the specified number of traces. When the batch size reaches 500, the N_{GE} decreases to 185, achieving the best attack performance. This indicates that our method requires considerably larger batch sizes than original methods to achieve improved performance. However, when the batch size exceeds 500, the N_{GE} begins to rise again. Therefore, to achieve optimal attack performance, the attacker must carefully select an appropriate batch size (e.g., 500, 1000) for training the neural network model.

Inspired by the research in [23], [31], we further investigate the impact of batch size on the diversity of positive and negative samples during training and the model's generalization ability. To this end, we propose two metrics for auxiliary analysis, namely Class Distribution Entropy (CDE) and Cosine Similarity of Gradient Vectors (CSGV). The CDE is used to measure the diversity of the class distribution in a mini-batch. Let the label set of the dataset be $Y = \{1, 2, \dots, C\}$, where C is the total number of classes in the current dataset. For a mini-batch containing N traces, let the number of traces with label y be n_i , then the probability of class i in the mini-batch

is:

$$p_i = \frac{n_i}{C} \quad (12)$$

The CDE is then defined as:

$$\text{CDE} = - \sum_{i=1}^C p_i \log(p_i) \quad (13)$$

A higher CDE indicates a more balanced class distribution with sufficient positive and negative samples. The CSGV is used to quantify the consistency of the gradient update direction during training. Let g_t be the gradient vector of the model's last layer parameters at the t -th iteration, where one iteration corresponds to a single mini-batch forward and backward update. Then, the CSGV over T consecutive iterations (with $T = 10$ in this study) is defined as:

$$\text{CSGV} = \frac{1}{T-1} \sum_{t=1}^{T-1} \text{sim}(g_t, g_{t+1}) \quad (14)$$

This metric characterizes the concentration of the gradient update path. A higher CSGV indicates that the gradient directions are excessively averaged, leading to a more deterministic update direction and potentially causing the model to converge to a flat or local minimum, which reduces its generalization ability and weakens its ability to learn key discriminative features.

Under the condition of fixed other hyperparameters, we calculate the CDE and the average CSGV of the last layer over 10 consecutive iterations for different batch sizes. Fig. 6 illustrates that the CDE gradually increases with larger batch sizes, due to the higher number of positive and negative samples in each mini-batch and the resulting more balanced class distribution. At the same time, as the batch size increases, the average cosine similarity between adjacent gradients also increases. When the batch size reaches 1000, the average cosine similarity between adjacent gradients reaches 0.34. An excessively large batch size causes the similarity distribution to become overly concentrated, reducing the diversity of gradient directions and decreasing the sensitivity of the loss function to difficult samples. Ultimately, the experimental results indicate that when the batch size is 500, the model achieves the best balance between positive-negative sample sufficiency and generalization ability.

Weight for SupCL-SCA. In Section III-B, we introduced the combination of ℓ_{cel} and ℓ_{sup} to jointly optimize the model's training. The goal of ℓ_{cel} is to improve the model's classification accuracy, while ℓ_{sup} extracts the similarity information between traces under supervised information, guiding the model to learn more discriminative feature representations. Since these two losses emphasize different optimization objectives, the relative weights w_{cel} and w_{sup} can significantly affect the model's performance. To investigate this impact, we evaluated how the model's performance varies with $w_{\text{cel}} : w_{\text{sup}}$ while keeping other hyperparameters fixed.

As shown in Fig. 7, when $w_{\text{cel}} : w_{\text{sup}}$ is too small or too large, the N_{GE} is high, indicating that overemphasizing ℓ_{sup} might cause the model to focus excessively on class differences while neglecting the optimization of classification accuracy. Conversely, overemphasizing ℓ_{cel} may result in

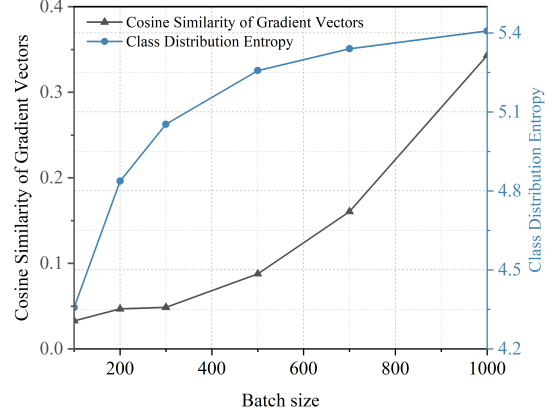


Fig. 6. Effect of batch size on gradient similarity and class distribution entropy

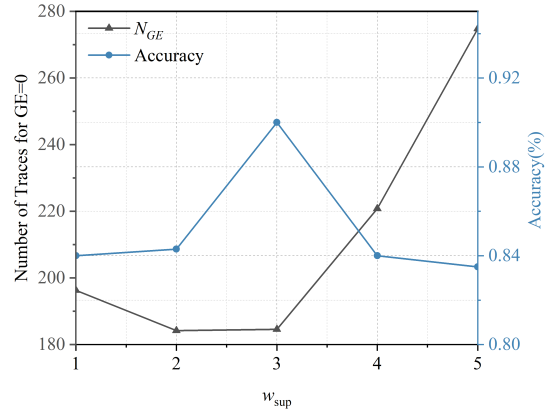


Fig. 7. Impact of w_{sup} when batch size is fixed

TABLE II
COMPARISON OF DIFFERENT METHODS

Method	CNN	Large Batch Size	Weight	N_{GE}
Original Method	✓	✗	✗	236
SupCL-SCA	✓	✓	✗	196
	✓	✓	✓	185

insufficient class separation, leading to a reduced ability to model key discriminative features and weakening the model's capacity to distinguish unseen samples. Therefore, an appropriate $w_{\text{cel}} : w_{\text{sup}}$ ratio enables a balanced trade-off between classification accuracy and class separation, thereby achieving better attack performance.

Finally, Table II shows a comparison of N_{GE} of SupCL-SCA the ASCAD^{sync} dataset under different batch sizes and loss function weight adjustments. The results demonstrate that by appropriately setting the large batch size and loss weights, SupCL-SCA can significantly reduce N_{GE} , confirming the key role of both factors in enhancing model performance.

C. Impacts of Hyper-parameters

To investigate the impact of the temperature coefficient τ and the constant ϵ on the attack performance of SupCL-SCA,

we conducted experiments on the ASCAD^{sync} dataset. As in Section III-B, we used the same network architecture and hyperparameters as in [21].

Temperature coefficient τ for SupCL-SCA. To investigate how the temperature coefficient τ affects the extraction of similarity information in SupCL-SCA, we first examine the gradient of ℓ_{sup} with respect to the feature vector z_i :

$$\frac{\partial \ell_{\text{sup}}}{\partial z_i} = \frac{1}{\tau N} \sum_{i=1}^N \left(- \sum_{p \in P(i)} w_p^+ \cdot \frac{\partial \text{sim}(z_i, z_p)}{\partial z_i} + \sum_{k=1}^N \mathbb{I}_{[k \neq i]} w_k^- \cdot \frac{\partial \text{sim}(z_i, z_k)}{\partial z_i} \right) \quad (15)$$

where $w_p^+ = \frac{\exp(\text{sim}(z_i, z_p)/\tau)}{S_+}$ and $w_k^- = \frac{\exp(\text{sim}(z_i, z_k)/\tau)}{S_-}$. As shown in equation (15), the temperature coefficient τ directly influences the similarity distribution between positive and negative samples by adjusting the exponential scaling of the similarity term $\text{sim}(z_i, z_j)$, and further regulates the overall magnitude of the gradient and its response to the similarity between positive and negative samples. To investigate the impact of τ on the model's performance, we fixed all hyperparameters except τ and evaluated how the model performance changes as τ varies.

As shown in Fig. 8, the experimental results indicate that the N_{GE} is minimized when $\tau = 0.07$, while both smaller and larger values of τ lead to reduced attack performance. Further analysis shows that smaller τ amplifies the gradient contrast between positive and negative samples, making the model more sensitive to slight differences in similarity, allowing it to quickly pull positive samples closer and push negative samples further apart. This accelerates the learning of the decision boundary, especially focusing on “hard samples,” i.e., those positive samples that are distant from the anchor point but belong to the same class, and negative samples that are close to the anchor point but belong to different classes. However, an overly small τ can lead to overly concentrated intra-class representations, weakening the model's generalization ability.

In contrast, larger τ causes the gradients to become more averaged, compressing the similarity differences between samples, which improves training stability and intra-class diversity, but may also diminish the model's focus on key discriminative features. Therefore, an appropriate value of τ (e.g., 0.07) allows the model to focus on both “easy-to-discriminate” and “hard-to-discriminate” negative samples during training, resulting in a smoother decision boundary and stronger attack performance.

Constant ϵ for SupCL-SCA. The constant ϵ in SupCL-SCA prevents gradient explosion when no samples from the same class are present in the mini-batch (i.e., when $S_{x_i}^+ = 0$). In this case, equation (6) would result in $\log 0$, causing numerical instability. To prevent numerical overflow, we modify ℓ_{x_i} as follows:

$$\ell_{x_i} = \log(S_{x_i}^-) - \log(\epsilon) \quad (16)$$

By introducing the compensation term $\epsilon \cdot \mathbb{I}_{[S_{x_i}^+ = 0]}$, we prevent the loss function from becoming undefined. As shown in Fig. 9, when ϵ is too small or too large, N_{GE} is higher. To

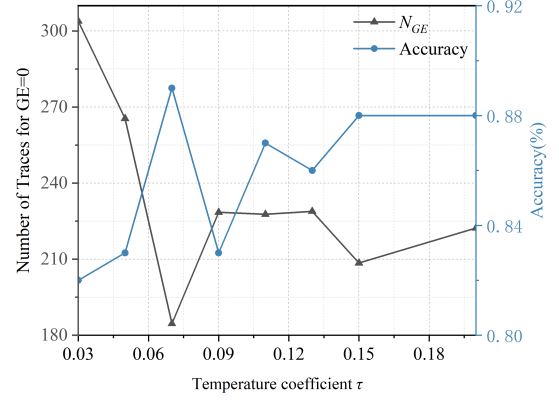


Fig. 8. Impact of temperature coefficient τ when constant ϵ is fixed

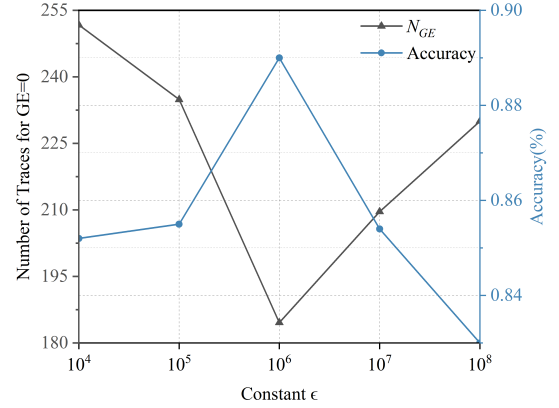


Fig. 9. Impact of constant ϵ when temperature coefficient τ is fixed

make the supervised contrastive loss ℓ_{sup} and cross-entropy loss ℓ_{cel} dimensionally consistent, we set ϵ to a large value (such as 10^6), which forces the model to minimize the loss by reducing the similarity of all negative samples when positive samples are missing, thus strengthening the model's ability to distinguish negative samples.

D. Comparison with existing works

To further evaluate the effectiveness of the proposed attack method, we compare it with the current state-of-the-art works, including [15], [19], [21], [32]. As described in Section III, our method does not require a specific architecture and is applicable to various network structures. In this section, we apply our method to the studies mentioned above and demonstrate that SupCL-SCA further improves performance while inheriting the advantages of the original network architecture.

When implementing SupCL-SCA, all hyperparameters, including network architecture, optimizer, learning rate, etc., were kept the same as in the original methods. Other experimental settings are summarized in Table III. And Table IV shows the results of SupCL-SCA and the original methods on the ASCAD^{sync}, ASCAD^{desync100}, AES_HD and CHES CTF 2018.

For ASCAD^{sync}, the results show that SupCL-SCA significantly outperforms the original methods across all four networks. Notably, in the method from [19], the number of traces

TABLE III
TRAINING PARAMETERS UNDER DIFFERENT EXPERIMENT SETUPS

Experiment Setup		[19]	[21]	[15]	[32]
ASCAD ^{sync}	Batch Size	1000	500	500	500
	Epoch	160	250	60	240
	$w_{\text{cel}}:w_{\text{sup}}$	1:1	1:3	10:1	4:1
ASCAD ^{desync100}	Batch Size	500	1000	1000	500
	Epoch	250	250	250	140
	$w_{\text{cel}}:w_{\text{sup}}$	1:1	1:3	2:1	2:1
AES_HD	Batch Size	1000	500	500	2000
	Epoch	150	300	250	130
	$w_{\text{cel}}:w_{\text{sup}}$	1:1	1:1	1:1	1:1
CHES CTF 2018	Batch Size	500	500	1000	1000
	Epoch	20	250	270	320
	$w_{\text{cel}}:w_{\text{sup}}$	1:1	1:1	1:1	1:1

TABLE IV
COMPARISON UNDER DIFFERENT EXPERIMENT SETUPS ON ASCAD, AES_HD, AND CHES CTF 2018

Experiment Setup			[19]	[21]	[15]	[32]
ASCAD ^{sync}	Original Method	N_{GE}	1156	236	95	126
		SR	100%	100%	100%	100%
	SupCL-SCA	N_{GE}	494	185	88	114
		SR	100%	100%	100%	100%
ASCAD ^{desync100}	Original Method	N_{GE}	8577	337	176	275
		SR	100%	100%	100%	100%
	SupCL-SCA	N_{GE}	1056	271	120	246
		SR	100%	100%	100%	100%
AES_HD	Original Method	N_{GE}	24965	1323	5202	1514
		SR	20%	100%	100%	100%
	SupCL-SCA	N_{GE}	15280	1183	4474	1122
		SR	100%	100%	100%	100%
CHES CTF 2018	Original Method	N_{GE}	—	—	336	—
		GE	156	210	0	146
		SR	0%	0%	80%	0%
		SR	0%	0%	80%	0%
	SupCL-SCA	N_{GE}	—	—	316	—
		GE	46	159	0	82
		SR	0%	0%	80%	0%
		SR	0%	0%	80%	0%

“—” denotes that the method fails to attack under a limited number of traces. SR denotes the success rate.

required for a successful attack decreased from 1156 to 494, a reduction of 57.2%. In the implementation of the method from [32], we observed large fluctuations in the number of traces needed for the attack. The average number of traces in the 10 experiments was higher than the reported results, but SupCL-SCA still achieved further performance improvement. For the current state-of-the-art method, B-CNN [15], SupCL-SCA required only 88 traces to converge to a GE of 0, a 7.4% improvement over the 95 traces required by the original method. These results demonstrate that SupCL-SCA adapts to various network architectures, improving performance while maintaining the advantages of the original network structure. Furthermore, SupCL-SCA requires almost no adjustment of the original parameter settings, such as optimizer, learning rate, or network architecture, and can directly use the original configurations while consistently improving the results.

For ASCAD^{desync100}, SupCL-SCA again improves the performance of the original methods while maintaining the advantages of the original network architecture. Specifically, in the method from [19], SupCL-SCA reduced the number of traces required for the attack from 8577 to 1056, a significant reduction of 87.7%. Even in the B-CNN [15] architecture, the number of traces required for the attack decreased from 176 to 120, a 31.8% reduction, further demonstrating the significant advantages of SupCL-SCA in countering desynchronization measures.

For the unprotected AES_HD, SupCL-SCA continues to show significant improvement over the original methods. We also analyzed the attack SR for different methods under the constraint of a limited number of traces. As shown in Table IV, in the method from [19], SupCL-SCA not only reduced the number of traces required for the attack by 38.8%, but also increased the attack SR from 20% to 100%, demonstrating stable and reliable attack capabilities. Notably, SupCL-SCA achieved a 100% SR across all network architectures, eliminating the random failure risk present in the original method under extreme cases.

Unlike the ASCAD and AES_HD datasets, the CHES CTF 2018 dataset involves a domain shift, where the training and attack traces originate from two different devices, and the trace length is considerably longer. Table IV presents our experimental results using identity leakage models to recover the secret key from the target device. Among the four methods, only the B-CNN [15] succeeds under the constraint of a limited number of attack traces. However, in 10 repeated experiments, the original B-CNN method does not always converge to a GE of 0 within 500 traces, and the number of required traces varies significantly. Therefore, we report the average number of traces required for successful attacks over 10 runs. Without compromising the attack SR, SupCL-SCA achieves further performance improvement under the B-CNN [15] architecture, recovering the correct key with only 316 traces. Since the methods in [19], [21] and [32] fail to succeed under limited trace conditions, we report their average GE instead. SupCL-SCA significantly outperforms the original methods in terms of average GE across all three network architectures.

TABLE V
COMPARISON UNDER CL-SCA AND SUPCL-SCA SETTINGS

Experiment Setup		[19]	[21]	[15]
ASCAD ^{sync}	CL-SCA	518	216	90
	SupCL-SCA	494	185	88
ASCAD ^{desync100}	CL-SCA	1828	332	124
	SupCL-SCA	1056	271	120

We also compare CL-SCA and SupCL-SCA on the ASCAD^{sync} and ASCAD^{desync100} datasets, as shown in Table V. The results indicate that the incorporation of supervised information particularly significant gains on the desynchronized dataset ASCAD^{desync100}. For the method in [19], the number of traces required for a successful attack is reduced from 1828 to 1056, a 42.2% decrease, suggesting that label information effectively mitigates the feature misalignment caused by temporal shifts. Even on the synchronized dataset

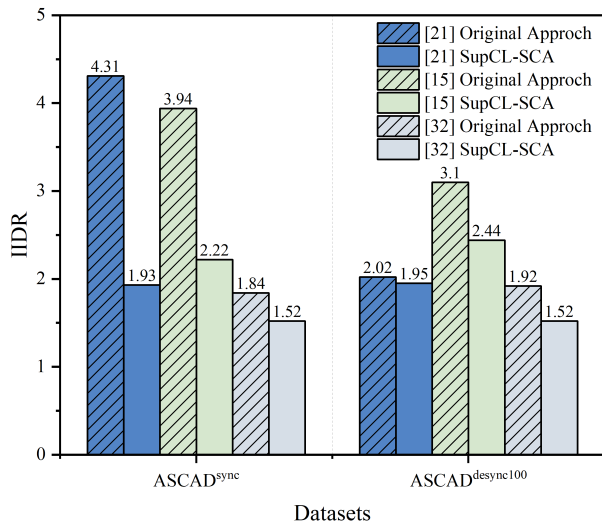


Fig. 10. IIDR comparison between original methods and SupCL-SCA on ASCAD^{sync} and ASCAD^{desync100}

ASCAD^{sync}, SupCL-SCA reduces the number of required traces by 18.3% under the network configuration of [21], further demonstrating the superiority of SupCL-SCA.

To validate that SupCL-SCA indeed brings samples of the same class closer in the embedding space while pushing apart those of different classes, we compute the IIDR values for the original methods from [21] [15] [32] and their corresponding SupCL-SCA variants on both ASCAD^{sync} and ASCAD^{desync100}. As shown in Fig. 10, on ASCAD^{sync}, the IIDR values for the methods in [21], [15], and [32] are reduced by 55.2%, 43.7%, and 17.3%, respectively. On ASCAD^{desync100}, SupCL-SCA also achieves consistent improvements ranging from 3.5% to 21.3%. These results demonstrate that by introducing a supervised contrastive learning mechanism, SupCL-SCA effectively enhances intra-class compactness and inter-class separability in the feature space under both synchronized and desynchronized scenarios.

V. CONCLUSION

In this paper, we propose a supervised contrastive learning-based profiled SCA, called SupCL-SCA. By leveraging supervised information to guide the extraction of discriminative features, this method not only enhances the intra-class compactness and inter-class separation but also resolves the issue in CL-SCA where samples of the same class may be incorrectly pushed apart. SupCL-SCA eliminates the need for complex data augmentation or a two-phase training process, significantly reducing the training complexity. The proposed SupCL-SCA loss can be seamlessly integrated into existing deep learning-based SCA frameworks, substantially improving the model's discriminative power and key recovery performance.

To further evaluate our method, experimental results demonstrate that SupCL-SCA exhibits superior performance across several public datasets, with especially significant improvements on desynchronized datasets compared to original methods. Additionally, we investigate the impact of hyperparam-

eters such as batch size, loss function weights, and temperature coefficient on model performance, providing valuable insights for further optimization and application of SupCL-SCA.

In the future, we plan to extend the concept of supervised contrastive learning to other SCA methods and explore its adaptability and performance improvement across diverse scenarios. We will also continue to optimize SupCL-SCA and further examine the impact of sampling strategies on model performance. This exploration will provide a better understanding of how similarity information between positive and negative samples influences training, leading to further optimization of SCA performance.

REFERENCES

- [1] D. Genkin, A. Shamir, and E. Tromer, "Rsa key extraction via low-bandwidth acoustic cryptanalysis," in *Advances in Cryptology—CRYPTO 2014: 34th Annual Cryptology Conference, Santa Barbara, CA, USA, August 17-21, 2014, Proceedings, Part I* 34. Springer, 2014, pp. 444–461.
- [2] D. Agrawal, B. Archambeault, J. R. Rao, and P. Rohatgi, "The em side-channel (s)," in *International workshop on cryptographic hardware and embedded systems*. Springer, 2002, pp. 29–45.
- [3] P. Kocher, J. Jaffe, and B. Jun, "Differential power analysis," in *Advances in Cryptology—CRYPTO'99: 19th Annual International Cryptology Conference Santa Barbara, California, USA, August 15–19, 1999 Proceedings* 19. Springer, 1999, pp. 388–397.
- [4] D. A. Osvik, A. Shamir, and E. Tromer, "Cache attacks and countermeasures: the case of aes," in *Topics in Cryptology—CT-RSA 2006: The Cryptographers' Track at the RSA Conference 2006, San Jose, CA, USA, February 13-17, 2005. Proceedings*. Springer, 2006, pp. 1–20.
- [5] P. C. Kocher, "Timing attacks on implementations of diffie-hellman, rsa, dss, and other systems," in *Advances in Cryptology—CRYPTO'96: 16th Annual International Cryptology Conference Santa Barbara, California, USA August 18–22, 1996 Proceedings* 16. Springer, 1996, pp. 104–113.
- [6] L. Lerman and O. Markowitch, "Efficient profiled attacks on masking schemes," *IEEE Transactions on Information Forensics and Security*, vol. 14, no. 6, pp. 1445–1454, 2018.
- [7] S. Paguada, L. Batina, I. Buhun, and I. Armendariz, "Playing with blocks: toward re-usable deep learning models for side-channel profiled attacks," *IEEE Transactions on Information Forensics and Security*, vol. 17, pp. 2835–2847, 2022.
- [8] M. O. Choudary and M. G. Kuhn, "Efficient, portable template attacks," *IEEE Transactions on Information Forensics and Security*, vol. 13, no. 2, pp. 490–501, 2017.
- [9] G. Hospodar, B. Gierlichs, E. De Mulder, I. Verbauwhede, and J. Vandewalle, "Machine learning in side-channel analysis: a first study," *Journal of Cryptographic Engineering*, vol. 1, no. 4, pp. 293–302, 2011.
- [10] L. Lerman, G. Bontempi, and O. Markowitch, "Side channel attack: an approach based on machine learning," *Center for Advanced Security Research Darmstadt*, vol. 29, 2011.
- [11] Y. Ou and L. Li, "Side-channel analysis attacks based on deep learning network," *Frontiers of Computer Science*, vol. 16, no. 2, p. 162303, 2022.
- [12] G. Zaid, L. Bossuet, F. Dassance, A. Habrard, and A. Venelli, "Ranking loss: Maximizing the success rate in deep learning side-channel analysis," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pp. 25–55, 2021.
- [13] Z. Xiao, C. Wang, J. Shen, Q. J. Wu, and D. He, "Less traces are all it takes: Efficient side-channel analysis on aes," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2024.
- [14] G. Zaid, L. Bossuet, A. Habrard, and A. Venelli, "Methodology for efficient cnn architectures in profiling attacks," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pp. 1–36, 2020.
- [15] P. Cao, C. Zhang, X. Lu, D. Gu, and S. Xu, "Improving deep learning based second-order side-channel analysis with bilinear cnn," *IEEE Transactions on Information Forensics and Security*, vol. 17, pp. 3863–3876, 2022.
- [16] A. Liu, A. Wang, S. Sun, C. Wei, Y. Ding, Y. Wang, and L. Zhu, "Cl-sca: A contrastive learning approach for profiled side-channel analysis," *IEEE Transactions on Information Forensics and Security*, 2025.

- [17] H. Maghrebi, T. Portigliatti, and E. Prouff, "Breaking cryptographic implementations using deep learning techniques," in *Security, Privacy, and Applied Cryptography Engineering: 6th International Conference, SPACE 2016, Hyderabad, India, December 14-18, 2016, Proceedings 6*. Springer, 2016, pp. 3–26.
- [18] E. Cagli, C. Dumas, and E. Prouff, "Convolutional neural networks with data augmentation against jitter-based countermeasures: Profiling attacks without pre-processing," in *Cryptographic Hardware and Embedded Systems—CHES 2017: 19th International Conference, Taipei, Taiwan, September 25-28, 2017, Proceedings*. Springer, 2017, pp. 45–68.
- [19] R. Benadjila, E. Prouff, R. Strullu, E. Cagli, and C. Dumas, "Deep learning for side-channel analysis and introduction to ascad database," *Journal of Cryptographic Engineering*, vol. 10, no. 2, pp. 163–188, 2020.
- [20] M. Jin, M. Zheng, H. Hu, and N. Yu, "An enhanced convolutional neural network in side-channel attacks and its visualization," *arXiv preprint arXiv:2009.08898*, 2020.
- [21] L. Wouters, V. Arribas, B. Gierlichs, and B. Preneel, "Revisiting a methodology for efficient cnn architectures in profiling attacks," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pp. 147–168, 2020.
- [22] F.-X. Standaert, T. G. Malkin, and M. Yung, "A unified framework for the analysis of side-channel key recovery attacks," in *Advances in Cryptology—EUROCRYPT 2009: 28th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Cologne, Germany, April 26-30, 2009. Proceedings 28*. Springer, 2009, pp. 443–461.
- [23] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A simple framework for contrastive learning of visual representations," in *International conference on machine learning*. Pmlr, 2020, pp. 1597–1607.
- [24] C. Yang, J. Zou, J. Wu, H. Xu, and S. Fan, "Supervised contrastive learning for recommendation," *Knowledge-Based Systems*, vol. 258, p. 109973, 2022.
- [25] F. Graf, C. Hofer, M. Niethammer, and R. Kwitt, "Dissecting supervised contrastive learning," in *International Conference on Machine Learning*. PMLR, 2021, pp. 3821–3830.
- [26] B. Gunel, J. Du, A. Conneau, and V. Stoyanov, "Supervised contrastive learning for pre-trained language model fine-tuning," *arXiv preprint arXiv:2011.01403*, 2020.
- [27] P. Khosla, P. Teterwak, C. Wang, A. Sarna, Y. Tian, P. Isola, A. Maschinot, C. Liu, and D. Krishnan, "Supervised contrastive learning," *Advances in neural information processing systems*, vol. 33, pp. 18 661–18 673, 2020.
- [28] Y. Zhang, S. Roller, and B. Wallace, "Mgnc-cnn: A simple approach to exploiting multiple word embeddings for sentence classification," *arXiv preprint arXiv:1603.00968*, 2016.
- [29] M. Seo, Y. Lee, and S. Kwak, "On the distribution of penultimate activations of classification networks," in *Uncertainty in Artificial Intelligence*. PMLR, 2021, pp. 1141–1151.
- [30] S. Picek, A. Heuser, A. Jovic, S. Bhasin, and F. Regazzoni, "The curse of class imbalance and conflicting metrics with machine learning for side-channel evaluations," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pp. 209–237, 2019.
- [31] J. T. Ash, S. Goel, A. Krishnamurthy, and D. Misra, "Investigating the role of negatives in contrastive representation learning," *arXiv preprint arXiv:2106.09943*, 2021.
- [32] S. Sun, H. Yu, A. Wang, C. Wei, Y. Ding, L. Zhu, and J. Chen, "Dual-path hybrid residual network for profiled side-channel analysis," *IEEE Transactions on Circuits and Systems II: Express Briefs*, 2024.



An Wang currently works as an Associate Professor with the School of Cyberspace Science and Technology, Beijing Institute of Technology, Beijing, China. His main research interests include side-channel analysis, cryptographic engineering, and implementation.



Yaoling Ding currently works as an Assistant Professor with the School of Cyberspace Science and Technology, Beijing Institute of Technology, Beijing, China. Her main research interests include side-channel analysis and block cipher analysis.



Annyu Liu is currently working towards a PhD degree with the School of Cyberspace Science and Technology at the Beijing Institute of Technology, Beijing, China. Her areas of research include side-channel analysis, machine learning techniques, and secure implementations of cryptographic algorithms.



Zheng Liu is currently working towards a PhD degree with the School of Cyberspace Science and Technology at the Beijing Institute of Technology, Beijing, China. His areas of research include side-channel analysis, post quantum cryptography, and fault injections.



Zoushaojie Jiang is currently pursuing the Master's degree with the School of Cyberspace Science and Technology at the Beijing Institute of Technology, Beijing, China. His research interests include side-channel analysis and machine learning techniques.



Yu Jing is currently a research assistant at the School of Cyberspace Science and Technology, Beijing Institute of Technology, China. Her main research interests include side-channel analysis and secure implementations of cryptographic algorithms.



Liehuang Zhu (Member, IEEE) is currently a Professor with the School of Cyberspace Science and Technology, Beijing Institute of Technology. He is selected into the Program for New Century Excellent Talents in University from the Ministry of Education, China. His research interests include the Internet of Things, cloud computing security, Internet, and mobile security.